



APRENDIZAJE AUTONOMO 1

Selección de Sistemas de Gestión Empresarial

PROGRAMACION ORIENTADA A OBJETOS

Presentado por:
JUAN ZHICAY

Tabla de contenido

- 1. Identificación del Proyecto 2
- 2. Objetivo Estratégico..... 2
- 3. Alcance Detallado del Proyecto 2
 - Gestión de Inventario Multimedia:..... 2
 - Motor de Búsqueda de Alto Rendimiento:..... 2
 - Simulación de Alta Demanda: 2
- 4. Estructura Funcional de los Módulos 2
 - A. Módulo de Administración de Contenido 2
 - B. Módulo de Usuario y Experiencia de Navegación 3
- 5. Estructura de Gestión 3
 - 1. Manejo de Concurrencia y Paralelismo:..... 3
 - 2. Gestión Dinámica de Memoria (Slices vs Arrays): 3
 - 3. Seguridad y Punteros:..... 3
- 6. Stack Tecnológico y Paquetes de Terceros..... 4
- 7. Configuración del Ecosistema (GitHub) 4

1. Identificación del Proyecto

- **Nombre del Sistema:** G-CINEMAPRO
 - **Lenguaje de Programación:** Go
 - **Paradigma Principal:** Programación Funcional y Concurrente.
-

2. Objetivo Estratégico

Diseñar y estructurar un sistema backend de alta disponibilidad para la gestión de bibliotecas de streaming. El sistema aplicará los fundamentos de la **Unidad 1**, tales como el manejo de memoria eficiente mediante punteros, la gestión de colecciones dinámicas con *Slices* y la optimización de búsquedas mediante *Hash Tables* (Maps), garantizando un procesamiento de datos inmutable y libre de efectos secundarios.

3. Alcance Detallado del Proyecto

El proyecto G-CINEMPRO no es una simple base de datos; es un motor de gestión que cubre:

- **Gestión de Inventario Multimedia:** Control total sobre el ciclo de vida de los contenidos (Películas, Series, Documentales).
 - **Motor de Búsqueda de Alto Rendimiento:** Implementación de algoritmos de filtrado que aprovechan la velocidad de Go.
 - **Simulación de Alta Demanda:** Uso de **Goroutines** para manejar múltiples peticiones de usuarios simultáneamente sin bloquear el sistema.
-

4. Estructura Funcional de los Módulos

- **A. Módulo de Administración de Contenido**

Este módulo se encarga de la persistencia lógica y validación de los datos.

- **Funcionalidad:** Registro, edición y baja lógica de títulos.
- **Especificación Técnica:** * Uso de **Structs** para definir entidades (Movie, Director, Genre).

- Implementación de **Setters con Punteros** para modificar el estado de los objetos de forma eficiente en memoria sin duplicar datos innecesariamente.
- Validación de datos mediante funciones puras antes de la inserción en el *Slice* principal.

- **B. Módulo de Usuario y Experiencia de Navegación**

Enfocado en la recuperación y visualización de la información.

- **Funcionalidad:** Buscador inteligente, categorización por género y sistema de "Mi Lista".
- **Especificación Técnica:**
 - **Filtrado Funcional:** Uso de funciones de orden superior para procesar *Slices* de películas.
 - **Optimización de Búsqueda:** Uso de **Mapas (Hash Tables)** para que la búsqueda por ID de película sea de tiempo constante , maximizando la velocidad de respuesta.

5. Estructura de Gestión

Para asegurar un desarrollo profesional, el sistema se regirá por estos tres pilares técnicos:

1. **Manejo de Concurrencia y Paralelismo:** Diferenciaremos ambos conceptos en el código. Usaremos la **concurrencia** para gestionar la entrada/salida de datos y el **paralelismo** para procesos pesados de filtrado si el hardware lo permite, utilizando el planificador de Go.
2. **Gestión Dinámica de Memoria (Slices vs Arrays):** Utilizaremos **Slices** para el catálogo, permitiendo un crecimiento dinámico del sistema. Aplicaremos la función `append` de forma controlada para entender cómo Go gestiona la capacidad y el re-dimensionamiento de los arreglos internos.
3. **Seguridad y Punteros:** Se emplearán punteros para el paso de grandes estructuras de datos entre funciones, evitando el costo de rendimiento de copiar objetos pesados, pero manteniendo la seguridad mediante el control de referencias.

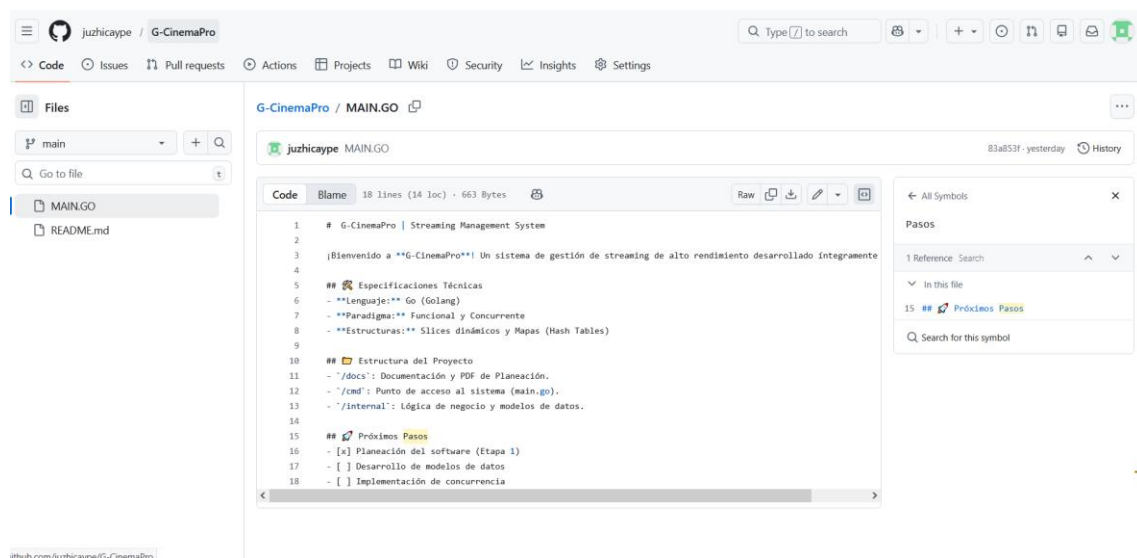
6. Stack Tecnológico y Paquetes de Terceros

- **Standard Library (Go):** fmt, errors, sync (para concurrencia), time.
 - **Paquetes de Terceros:**
 - **Gokit/endpoint:** Para estructurar el sistema como un microservicio (si se desea escalar).
 - **Testify:** Para realizar pruebas unitarias funcionales y asegurar que cada módulo responda correctamente.
-

7. Configuración del Ecosistema (GitHub)

El repositorio estará organizado bajo el estándar de la comunidad de Go:

- /cmd: Punto de entrada de la aplicación (main.go).
- /internal/models: Definición de estructuras y métodos.
- /internal/service: Lógica funcional de negocio.
- README.md: Documentación técnica, guía de instalación y diagramas de arquitectura.



```
1 # G-CinemaPro | Streaming Management System
2
3 |Bienvenido a **G-CinemaPro**| Un sistema de gestión de streaming de alto rendimiento desarrollado íntegramente
4
5 ## 📄 Especificaciones Técnicas
6 - **Lenguaje:** Go (Golang)
7 - **Paradigma:** Funcional y Concurrente
8 - **Estructuras:** Slices dinámicos y Mapas (Hash Tables)
9
10 ## 📁 Estructura del Proyecto
11 - './docs': Documentación y PDF de Planeación.
12 - './cmd': Punto de acceso al sistema (main.go).
13 - './internal': Lógica de negocio y modelos de datos.
14
15 ## 🚀 Próximos Pasos
16 - [x] Planeación del software (Etapas 1)
17 - [ ] Desarrollo de modelos de datos
18 - [ ] Implementación de concurrencia
```